Sum Of Linked Lists

AlgoExpert · Medium · Linked Lists

The problem

- Lists store digits least-significant first; return their sum in the same format.

Intuition

- Add digit by digit like grade school, carrying into the next node.

Brute force vs optimal

- Convert to integers, add, convert back → overflows for long lists
- Digit-by-digit with carry → O(max(n, m))
- Dummy head simplifies building

Approach

- 1. Walk both lists together
- 2. Sum digits plus carry
- 3. Create a node with the ones digit
- 4. Carry the rest

Worked example

1742 (reversed)



549 (reversed)



result



2 + 9 + carry → digit 1, carry 1

Worked example

1742 (reversed)



549 (reversed)



result



4 + 4 + carry → digit 9, carry 0

Worked example

1742 (reversed)



549 (reversed)



result



7 + 5 + carry → digit 2, carry 1

Worked example

1742 (reversed)



549 (reversed)



result



1 + 0 + carry → digit 2, carry 0

Java solution

```java
public class SumOfLinkedLists {
    public static LinkedListNode sumOfLinkedLists(LinkedListNode a, LinkedListNode b) {
        LinkedListNode dummy = new LinkedListNode(0), cur = dummy;
        int carry = 0;
        while (a != null || b != null || carry != 0) {
            int s = carry + (a == null ? 0 : a.value) + (b == null ? 0 : b.value);
            cur.next = new LinkedListNode(s % 10);
            cur = cur.next;
            carry = s / 10;
            if (a != null) a = a.next;
            if (b != null) b = b.next;
        }
        return dummy.next;
    }
}
```

Complexity

- Time: O(max(n, m))
- Space: O(max(n, m))

Edge cases & pitfalls

- Final carry creates an extra node
- Lists of different lengths
- Use a dummy head

Interview tips & follow-ups

- Follow-up: digits stored most-significant first (use stacks or reverse)

Recap

- Add digit by digit like grade school, carrying into the next node.
- Complexity: O(max(n, m))